

МИНИСТЕРСТВО ЛЕСНОГО ХОЗЯЙСТВА И ОХРАНЫ ОБЪЕКТОВ ЖИВОТНОГО МИРА
НИЖЕГОРОДСКОЙ ОБЛАСТИ

Государственное бюджетное профессиональное образовательное
учреждение Нижегородской области

«КРАСНОБАКОВСКИЙ ЛЕСНОЙ КОЛЛЕДЖ»

(ГБПОУ НО «КБЛК»)

День 14.

**Тема: «РАЗРАБОТКА СЕРВИСНОЙ ЧАСТИ
ПРОГРАММЫ»**

Выполнила: Потанина Н.А.,

Студентка группы: 24-ИС.

2026г.

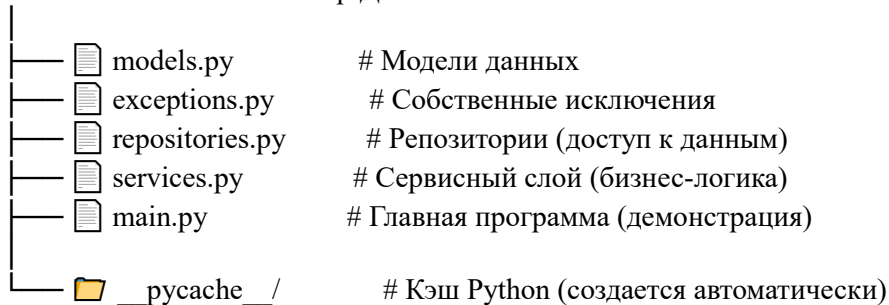
Цель работы:

Изучить принципы построения сервисного слоя (Service Layer) в приложении, освоить паттерны **Repository**, **Service**, **Dependency Injection**, научиться проектировать бизнес-логику, отделённую от инфраструктурного кода, и покрывать её юнит-тестами.

Вариант 2: Система бронирования отелей

Структура проекта:

C:/Users/Student237/Desktop/День 14/



models.py - Модели данных для системы бронирования отелей

```
from datetime import datetime, date
from dataclasses import dataclass
from typing import Optional, List
from enum import Enum
from decimal import Decimal
```

```
class RoomType(str, Enum):
    """Типы номеров."""
    STANDARD = "standard"
    BUSINESS = "business"
    SUITE = "suite"
    DELUXE = "deluxe"
```

```
class BookingStatus(str, Enum):
    """Статусы бронирования."""
    ACTIVE = "active"
    CANCELLED = "cancelled"
    COMPLETED = "completed"
    PENDING = "pending"
```

```
@dataclass
class Room:
    """Модель номера."""
    id: int
    hotel_name: str
    room_number: str
    room_type: RoomType
    price_per_night: Decimal
```

```

max_guests: int
is_available: bool = True
amenities: List[str] = None

def __post_init__(self):
    if self.amenities is None:
        self.amenities = []

@dataclass
class Booking:
    """Модель бронирования."""
    id: int
    room_id: int
    guest_name: str
    guest_email: str
    guest_phone: str
    check_in: date
    check_out: date
    total_price: Decimal
    status: BookingStatus = BookingStatus.PENDING
    created_at: datetime = None
    cancelled_at: datetime = None

    def __post_init__(self):
        if self.created_at is None:
            self.created_at = datetime.now()

```

1.models.py

exceptions.py - Собственные исключения

```

class HotelBookingError(Exception):
    """Базовое исключение."""
    pass

class ValidationError(HotelBookingError):
    """Ошибка валидации."""
    pass

class RoomNotFoundError(HotelBookingError):
    """Номер не найден."""
    def __init__(self, room_id: int):
        self.room_id = room_id
        super().__init__(f"Номер с ID {room_id} не найден")

class BookingNotFoundError(HotelBookingError):
    """Бронирование не найдено."""
    def __init__(self, booking_id: int):
        self.booking_id = booking_id
        super().__init__(f"Бронирование с ID {booking_id} не найдено")

class RoomNotAvailableError(HotelBookingError):

```

```

"""Номер недоступен."""
def __init__(self, room_id: int, check_in: str, check_out: str):
    super().__init__(f'Номер {room_id} не доступен на даты {check_in} - {check_out}')

```

```

class BookingCancellationError(HotelBookingError):
    """Ошибка отмены."""
    pass

```

```

class BusinessRuleViolationError(HotelBookingError):
    """Нарушение бизнес-правил."""
    Pass

```

2. exceptions.py

repositories.py - Репозитории для работы с данными

```

from typing import List, Optional, Dict
from datetime import date, datetime # <-- ДОБАВЛЯЕМ datetime
from collections import defaultdict
import threading

```

```

from models import Room, Booking, BookingStatus
from exceptions import RoomNotFoundError, BookingNotFoundError

```

```

class InMemoryRoomRepository:
    """In-Memory репозиторий номеров."""

```

```

    def __init__(self):
        self._rooms: Dict[int, Room] = {}
        self._next_id = 1
        self._lock = threading.Lock()

```

```

    def add(self, room: Room) -> Room:
        with self._lock:
            room.id = self._next_id
            self._rooms[room.id] = room
            self._next_id += 1
            return room

```

```

    def get_by_id(self, room_id: int) -> Optional[Room]:
        with self._lock:
            return self._rooms.get(room_id)

```

```

    def update(self, room: Room) -> Room:
        with self._lock:
            if room.id not in self._rooms:
                raise RoomNotFoundError(room.id)
            self._rooms[room.id] = room
            return room

```

```

    def get_all(self, limit: int = 100, offset: int = 0) -> List[Room]:
        with self._lock:
            rooms = list(self._rooms.values())
            return rooms[offset:offset + limit]

```

```

def find_available(self, **kwargs) -> List[Room]:
    """Поиск доступных номеров по критериям."""
    with self._lock:
        result = []
        hotel_name = kwargs.get('hotel_name')
        room_type = kwargs.get('room_type')
        max_guests = kwargs.get('max_guests')
        min_price = kwargs.get('min_price')
        max_price = kwargs.get('max_price')

        for room in self._rooms.values():
            if not room.is_available:
                continue

            if hotel_name and hotel_name.lower() not in room.hotel_name.lower():
                continue
            if room_type and room.room_type != room_type:
                continue
            if max_guests and room.max_guests < max_guests:
                continue
            if min_price and room.price_per_night < min_price:
                continue
            if max_price and room.price_per_night > max_price:
                continue

            result.append(room)
        return result

```

```

def update_availability(self, room_id: int, is_available: bool) -> None:
    with self._lock:
        room = self._rooms.get(room_id)
        if room:
            room.is_available = is_available

```

```

class InMemoryBookingRepository:
    """In-Memory репозиторий бронирований."""

```

```

    def __init__(self):
        self._bookings: Dict[int, Booking] = {}
        self._next_id = 1
        self._lock = threading.Lock()
        self._room_bookings: Dict[int, List[int]] = defaultdict(list)
        self._guest_bookings: Dict[str, List[int]] = defaultdict(list)

    def add(self, booking: Booking) -> Booking:
        with self._lock:
            booking.id = self._next_id
            self._bookings[booking.id] = booking
            self._room_bookings[booking.room_id].append(booking.id)
            self._guest_bookings[booking.guest_email].append(booking.id)
            self._next_id += 1
            return booking

    def get_by_id(self, booking_id: int) -> Optional[Booking]:
        with self._lock:
            return self._bookings.get(booking_id)

```

```

def update(self, booking: Booking) -> Booking:
    with self._lock:
        if booking.id not in self._bookings:
            raise BookingNotFoundError(booking.id)
        self._bookings[booking.id] = booking
        return booking

def get_by_room_and_dates(self, room_id: int, check_in: date, check_out: date) -> List[Booking]:
    """Получить пересекающиеся бронирования для номера."""
    with self._lock:
        result = []
        for booking_id in self._room_bookings.get(room_id, []):
            booking = self._bookings.get(booking_id)
            if not booking or booking.status == BookingStatus.CANCELLED:
                continue
            # Проверка пересечения дат
            if not (booking.check_out <= check_in or booking.check_in >= check_out):
                result.append(booking)
        return result

def get_by_guest_email(self, email: str) -> List[Booking]:
    with self._lock:
        return [
            self._bookings[bid]
            for bid in self._guest_bookings.get(email, [])
            if bid in self._bookings
        ]

def cancel(self, booking_id: int, reason: Optional[str] = None) -> Booking:
    with self._lock:
        booking = self._bookings.get(booking_id)
        if not booking:
            raise BookingNotFoundError(booking_id)

        if booking.status == BookingStatus.CANCELLED:
            raise ValueError("Бронирование уже отменено")

        if booking.status == BookingStatus.COMPLETED:
            raise ValueError("Завершенное бронирование нельзя отменить")

        booking.status = BookingStatus.CANCELLED
        booking.cancelled_at = datetime.now() # <-- теперь datetime импортирован
        return booking

```

3. repositories.py

services.py - Сервисный слой

```

import logging
from typing import List, Optional, Dict
from datetime import date, timedelta
from decimal import Decimal

from models import Room, Booking, BookingStatus
from exceptions import (
    RoomNotFoundError, BookingNotFoundError, RoomNotAvailableError,
    ValidationError, BookingCancellationError, BusinessRuleViolationError

```

)

```
logger = logging.getLogger(__name__)
```

```
class WeatherService:
```

```
    """Сервис погоды (внешний API)."""
```

```
    def __init__(self):
```

```
        self._cache: Dict[str, Dict] = {}
```

```
        logger.info("WeatherService инициализирован")
```

```
    def get_weather_forecast(self, city: str, target_date: date) -> Dict:
```

```
        """Получить прогноз погоды (имитация)."""
```

```
        cache_key = f'{city}_{target_date.isoformat()}'
```

```
        if cache_key in self._cache:
```

```
            return self._cache[cache_key]
```

```
        # Имитация API
```

```
        forecast = {
```

```
            "city": city,
```

```
            "date": target_date.isoformat(),
```

```
            "temperature": 22.5,
```

```
            "weather": "sunny",
```

```
            "precipitation": 0
```

```
        }
```

```
        self._cache[cache_key] = forecast
```

```
        return forecast
```

```
    def get_cancellation_risk(self, city: str, target_date: date) -> float:
```

```
        """Оценить риск отмены."""
```

```
        try:
```

```
            forecast = self.get_weather_forecast(city, target_date)
```

```
            risk = 0.0
```

```
            if forecast.get('precipitation', 0) > 5:
```

```
                risk += 0.3
```

```
            if forecast.get('temperature', 20) > 35:
```

```
                risk += 0.2
```

```
            return min(risk, 1.0)
```

```
        except Exception:
```

```
            return 0.5
```

```
class HotelBookingService:
```

```
    """Сервис бронирования отелей."""
```

```
    def __init__(
```

```
        self,
```

```
        room_repository,
```

```
        booking_repository,
```

```
        weather_service: Optional[WeatherService] = None
```

```
):
```

```
    self.room_repo = room_repository
```

```
    self.booking_repo = booking_repository
```

```
    self.weather_service = weather_service or WeatherService()
```

```
    logger.info("HotelBookingService инициализирован")
```

```

def add_room(self, room_data) -> Room:
    """Добавить новый номер."""
    logger.info(f'Добавление номера: {room_data.hotel_name}, {room_data.room_number}')

    room = Room(
        id=0,
        hotel_name=room_data.hotel_name,
        room_number=room_data.room_number,
        room_type=room_data.room_type,
        price_per_night=room_data.price_per_night,
        max_guests=room_data.max_guests,
        is_available=True,
        amenities=room_data.amenities or []
    )

    result = self.room_repo.add(room)
    logger.info(f'Номер добавлен: ID={result.id}')
    return result

def search_available_rooms(self, search_params) -> List[Room]:
    """Поиск доступных номеров."""
    # Проверка дат
    if hasattr(search_params, 'check_in') and search_params.check_in:
        if hasattr(search_params, 'check_out') and search_params.check_out:
            if search_params.check_out <= search_params.check_in:
                raise ValidationError("Дата выезда должна быть позже даты заезда")

    # Поиск
    rooms = self.room_repo.find_available(
        hotel_name=getattr(search_params, 'hotel_name', None),
        room_type=getattr(search_params, 'room_type', None),
        max_guests=getattr(search_params, 'max_guests', None),
        min_price=getattr(search_params, 'min_price', None),
        max_price=getattr(search_params, 'max_price', None)
    )

    # Проверка на конкретные даты
    if hasattr(search_params, 'check_in') and search_params.check_in:
        if hasattr(search_params, 'check_out') and search_params.check_out:
            available = []
            for room in rooms:
                existing = self.booking_repo.get_by_room_and_dates(
                    room.id,
                    search_params.check_in,
                    search_params.check_out
                )
                if not existing:
                    available.append(room)
            rooms = available

    logger.info(f'Найдено {len(rooms)} номеров')
    return rooms

def create_booking(self, booking_data) -> Booking:
    """Создать бронирование."""
    logger.info(f'Создание бронирования: номер {booking_data.room_id}')

```



```

# Проверка номера
room = self.room_repo.get_by_id(booking_data.room_id)
if not room:
    raise RoomNotFoundError(booking_data.room_id)

if not room.is_available:
    raise RoomNotAvailableError(
        booking_data.room_id,
        booking_data.check_in.isoformat(),
        booking_data.check_out.isoformat()
    )

# Проверка занятости
existing = self.booking_repo.get_by_room_and_dates(
    booking_data.room_id,
    booking_data.check_in,
    booking_data.check_out
)
if existing:
    raise RoomNotAvailableError(
        booking_data.room_id,
        booking_data.check_in.isoformat(),
        booking_data.check_out.isoformat()
    )

# Бизнес-правила
nights = (booking_data.check_out - booking_data.check_in).days
if nights > 30:
    raise BusinessRuleViolationError(f"Максимум 30 ночей, запрошено {nights}")

# Проверка активных бронирований гостя
guest_bookings = self.booking_repo.get_by_guest_email(booking_data.guest_email)
active = [b for b in guest_bookings if b.status == BookingStatus.ACTIVE]
if len(active) >= 3:
    raise BusinessRuleViolationError(
        f"Гость уже имеет {len(active)} активных бронирований"
    )

# Расчет стоимости
total_price = room.price_per_night * nights

# Создание бронирования
booking = Booking(
    id=0,
    room_id=booking_data.room_id,
    guest_name=booking_data.guest_name,
    guest_email=booking_data.guest_email,
    guest_phone=booking_data.guest_phone,
    check_in=booking_data.check_in,
    check_out=booking_data.check_out,
    total_price=total_price,
    status=BookingStatus.ACTIVE
)

result = self.booking_repo.add(booking)
self.room_repo.update_availability(booking_data.room_id, False)

```

```

logger.info(f"Бронирование создано: ID={result.id}, Сумма={result.total_price}")
return result

def cancel_booking(self, booking_id: int, reason: Optional[str] = None) -> Booking:
    """Отменить бронирование."""
    logger.info(f"Отмена бронирования: {booking_id}")

    booking = self.booking_repo.get_by_id(booking_id)
    if not booking:
        raise BookingNotFoundError(booking_id)

    if booking.status == BookingStatus.COMPLETED:
        raise BookingCancellationError("Завершенное бронирование нельзя отменить")

    if booking.status == BookingStatus.CANCELLED:
        raise BookingCancellationError("Бронирование уже отменено")

    # Расчет штрафа
    today = date.today()
    days_until = (booking.check_in - today).days
    penalty = Decimal('0')

    if days_until < 3:
        penalty = booking.total_price * Decimal('0.5')
        logger.info(f"Штраф: {penalty} руб. (менее 3 дней)")
    elif days_until < 7:
        penalty = booking.total_price * Decimal('0.25')
        logger.info(f"Штраф: {penalty} руб. (менее 7 дней)")

    result = self.booking_repo.cancel(booking_id, reason)
    self.room_repo.update_availability(booking.room_id, True)

    logger.info(f"Бронирование {booking_id} отменено, штраф: {penalty}")
    return result

def complete_booking(self, booking_id: int) -> Booking:
    """Завершить бронирование."""
    logger.info(f"Завершение бронирования: {booking_id}")

    booking = self.booking_repo.get_by_id(booking_id)
    if not booking:
        raise BookingNotFoundError(booking_id)

    if booking.status != BookingStatus.ACTIVE:
        raise ValidationError(f"Бронирование {booking_id} не активно")

    booking.status = BookingStatus.COMPLETED
    result = self.booking_repo.update(booking)
    self.room_repo.update_availability(booking.room_id, True)

    logger.info(f"Бронирование {booking_id} завершено")
    return result

def get_guest_bookings(self, email: str) -> List[Booking]:
    """Получить бронирования гостя."""
    return self.booking_repo.get_by_guest_email(email)

```

```

def get_weather_recommendation(self, city: str, check_in: date, check_out: date) -> Dict:
    """Получить рекомендации по погоде."""
    recommendations = {
        "city": city,
        "overall_risk": 0.0,
        "recommendation": "Погода благоприятная"
    }

    total_risk = 0.0
    days = (check_out - check_in).days

    for i in range(days):
        current_date = check_in + timedelta(days=i)
        risk = self.weather_service.get_cancellation_risk(city, current_date)
        total_risk += risk

    if days > 0:
        avg_risk = total_risk / days
        recommendations["overall_risk"] = avg_risk

    if avg_risk > 0.7:
        recommendations["recommendation"] = "Высокий риск, рекомендуем перенести поездку"
    elif avg_risk > 0.4:
        recommendations["recommendation"] = "Умеренный риск, возьмите зонт"

    return recommendations

```

4. services.py

main.py - Главная программа

```

import logging
import sys
from datetime import date, timedelta
from decimal import Decimal

```

```

from models import RoomType
from repositories import InMemoryRoomRepository, InMemoryBookingRepository
from services import HotelBookingService, WeatherService

```

Простые классы для данных (без Pydantic)

class RoomCreate:

```

    def __init__(self, hotel_name, room_number, room_type, price_per_night, max_guests,
amenities=None):
        self.hotel_name = hotel_name
        self.room_number = room_number
        self.room_type = room_type
        self.price_per_night = price_per_night
        self.max_guests = max_guests
        self.amenities = amenities or []

```

class BookingCreate:

```

    def __init__(self, room_id, guest_name, guest_email, guest_phone, check_in, check_out):
        self.room_id = room_id
        self.guest_name = guest_name

```

```

self.guest_email = guest_email
self.guest_phone = guest_phone
self.check_in = check_in
self.check_out = check_out

```

```

class RoomSearch:

```

```

    def __init__(self, hotel_name=None, room_type=None, check_in=None, check_out=None,
                  max_guests=None, min_price=None, max_price=None):
        self.hotel_name = hotel_name
        self.room_type = room_type
        self.check_in = check_in
        self.check_out = check_out
        self.max_guests = max_guests
        self.min_price = min_price
        self.max_price = max_price

```

```

def setup_logging():

```

```

    """Настройка логирования."""
    logging.basicConfig(
        level=logging.INFO,
        format='%(asctime)s - %(levelname)s - %(message)s',
        datefmt='%H:%M:%S'
    )

```

```

def main():

```

```

    """Главная функция."""

```

```

    # Настройка логирования
    setup_logging()

```

```

    print("\n" + "="*70)
    print("  СИСТЕМА БРОНИРОВАНИЯ ОТЕЛЕЙ".center(70))
    print("="*70 + "\n")

```

```

    # Создание зависимостей

```

```

    print("  Инициализация компонентов...")
    room_repo = InMemoryRoomRepository()
    booking_repo = InMemoryBookingRepository()
    weather_service = WeatherService()
    service = HotelBookingService(room_repo, booking_repo, weather_service)
    print("  Сервис готов к работе\n")

```

```

    # =====
    # 1. ДОБАВЛЕНИЕ НОМЕРОВ
    # =====

```

```

    print("1  ДОБАВЛЕНИЕ НОМЕРОВ")
    print("-"*50)

```

```

    room1 = service.add_room(RoomCreate(
        hotel_name="Grand Hotel",
        room_number="101",
        room_type=RoomType.STANDARD,
        price_per_night=Decimal("5000.00"),
        max_guests=2,

```

```

        amenities=["wifi", "tv"]
    ))
    print(f"          ✓ Номер #{room1.id}: {room1.hotel_name} - {room1.room_number}
({room1.price_per_night} руб/ночь)")

    room2 = service.add_room(RoomCreate(
        hotel_name="Grand Hotel",
        room_number="201",
        room_type=RoomType.SUITE,
        price_per_night=Decimal("15000.00"),
        max_guests=4,
        amenities=["wifi", "tv", "jacuzzi"]
    ))
    print(f"          ✓ Номер #{room2.id}: {room2.hotel_name} - {room2.room_number}
({room2.price_per_night} руб/ночь)")

    room3 = service.add_room(RoomCreate(
        hotel_name="Beach Resort",
        room_number="A1",
        room_type=RoomType.DELUXE,
        price_per_night=Decimal("25000.00"),
        max_guests=6,
        amenities=["wifi", "pool", "spa"]
    ))
    print(f"          ✓ Номер #{room3.id}: {room3.hotel_name} - {room3.room_number}
({room3.price_per_night} руб/ночь)\n")

# =====
# 2. ПОИСК НОМЕРОВ
# =====
print("\u25b6 ПОИСК ДОСТУПНЫХ НОМЕРОВ")
print("-"*50)

today = date.today()
check_in = today + timedelta(days=7)
check_out = check_in + timedelta(days=3)

search = RoomSearch(
    hotel_name="Grand",
    check_in=check_in,
    check_out=check_out,
    max_guests=2
)

available = service.search_available_rooms(search)
print(f" Найдено номеров: {len(available)}")
for room in available:
    print(f" • {room.hotel_name} - {room.room_number} ({room.room_type.value})")
print()

# =====
# 3. СОЗДАНИЕ БРОНИРОВАНИЯ
# =====
print("\u25b6 СОЗДАНИЕ БРОНИРОВАНИЯ")
print("-"*50)

booking1 = service.create_booking(BookingCreate(

```

```
room_id=room1.id,
guest_name="Иван Петров",
guest_email="ivan@mail.ru",
guest_phone="+7 912 345-67-89",
check_in=check_in,
check_out=check_out
))
print(f" ✓ Бронирование #{booking1.id}")
print(f"   Гость: {booking1.guest_name}")
print(f"   Даты: {booking1.check_in} → {booking1.check_out}")
print(f"   Сумма: {booking1.total_price} руб.")
print(f"   Статус: {booking1.status.value}\n")
```

```
# =====
# 4. ОШИБКА: ЗАНЯТЫЙ НОМЕР
# =====
```

```
print(f"❏ ПОПЫТКА БРОНИРОВАНИЯ ЗАНЯТОГО НОМЕРА")
print("-"*50)
try:
    service.create_booking(BookingCreate(
        room_id=room1.id,
        guest_name="Петр Сидоров",
        guest_email="petr@mail.ru",
        guest_phone="+7 913 456-78-90",
        check_in=check_in,
        check_out=check_out
    ))
except Exception as e:
    print(f" ❌ Ошибка (ожидаемая): {e}\n")
```

```
# =====
# 5. ОТМЕНА БРОНИРОВАНИЯ
# =====
```

```
print(f"❏ ОТМЕНА БРОНИРОВАНИЯ")
print("-"*50)
```

```
# Создаем бронирование на завтра для штрафа
booking2 = service.create_booking(BookingCreate(
    room_id=room2.id,
    guest_name="Анна Смирнова",
    guest_email="anna@mail.ru",
    guest_phone="+7 914 567-89-01",
    check_in=today + timedelta(days=1),
    check_out=today + timedelta(days=2)
))
print(f" Создано бронирование #{booking2.id} на завтра")

cancelled = service.cancel_booking(booking2.id, "Изменение планов")
print(f" ✓ Бронирование #{cancelled.id} отменено")
print(f"   Статус: {cancelled.status.value}\n")
```

```
# =====
# 6. ПОГОДНЫЕ РЕКОМЕНДАЦИИ
# =====
```

```
print(f"❏ ПОГОДНЫЕ РЕКОМЕНДАЦИИ")
print("-"*50)
```

```

weather = service.get_weather_recommendation("Moscow", check_in, check_out)
print(f" Город: {weather['city']}")
print(f" Риск: {weather['overall_risk']:.2f}")
print(f" Рекомендация: {weather['recommendation']}\n")

```

```

# =====
# 7. БРОНИРОВАНИЯ ГОСТЯ
# =====

```

```

print(f"📅 БРОНИРОВАНИЯ ГОСТЯ")
print("-"*50)

```

```

guest_bookings = service.get_guest_bookings("ivan@mail.ru")
print(f" Найдено бронирований: {len(guest_bookings)}")
for b in guest_bookings:
    print(f" •#{b.id}: {b.guest_name}, {b.total_price} руб., {b.status.value}")
print()

```

```

# =====
# ИТОГИ
# =====

```

```

print("="*70)
print("✅ ВСЕ ОПЕРАЦИИ ВЫПОЛНЕНЫ УСПЕШНО".center(70))
print("="*70)
print("\n📊 ИТОГ:")
print(f" • Добавлено номеров: 3")
print(f" • Создано бронирований: 2")
print(f" • Отменено бронирований: 1")
print(f" • Активных бронирований: 1")
print("\n🏠 Работа программы завершена\n")

```

```

if __name__ == "__main__":
    main()

```

5. main.py

```

=====
🏠 СИСТЕМА БРОНИРОВАНИЯ ОТЕЛЕЙ
=====

📁 Инициализация компонентов...
12:36:34 - INFO - WeatherService инициализирован
12:36:34 - INFO - HotelBookingService инициализирован
✅ Сервис готов к работе

1 📅 ДОБАВЛЕНИЕ НОМЕРОВ
-----
12:36:34 - INFO - Добавление номера: Grand Hotel, 101
12:36:34 - INFO - Номер добавлен: ID=1
✅ Номер #1: Grand Hotel - 101 (5000.00 руб/ночь)
12:36:34 - INFO - Добавление номера: Grand Hotel, 201
12:36:34 - INFO - Номер добавлен: ID=2
✅ Номер #2: Grand Hotel - 201 (15000.00 руб/ночь)
12:36:34 - INFO - Добавление номера: Beach Resort, A1
12:36:34 - INFO - Номер добавлен: ID=3
✅ Номер #3: Beach Resort - A1 (25000.00 руб/ночь)

2 📅 ПОИСК ДОСТУПНЫХ НОМЕРОВ
-----
12:36:34 - INFO - Найдено 2 номеров
Найдено номеров: 2
• Grand Hotel - 101 (standard)
• Grand Hotel - 201 (suite)

```

```

3 ☐ СОЗДАНИЕ БРОНИРОВАНИЯ
-----
12:36:34 - INFO - Создание бронирования: номер 1
12:36:34 - INFO - Бронирование создано: ID=1, Сумма=15000.00
✓ Бронирование #1
  Гость: Иван Петров
  Даты: 2026-06-29 → 2026-07-02
  Сумма: 15000.00 руб.
  Статус: active

4 ☐ ПОПЫТКА БРОНИРОВАНИЯ ЗАНЯТОГО НОМЕРА
-----
12:36:34 - INFO - Создание бронирования: номер 1
  ✗ Ошибка (ожидаемая): Номер 1 не доступен на даты 2026-06-29 - 2026-07-02

5 ☐ ОТМЕНА БРОНИРОВАНИЯ
-----
12:36:34 - INFO - Создание бронирования: номер 2
12:36:34 - INFO - Бронирование создано: ID=2, Сумма=15000.00
  Создано бронирование #2 на завтра
12:36:34 - INFO - Отмена бронирования: 2
12:36:34 - INFO - Штраф: 7500.000 руб. (менее 3 дней)
12:36:34 - INFO - Бронирование 2 отменено, штраф: 7500.000
✓ Бронирование #2 отменено
  Статус: cancelled

6 ☐ ПОГОДНЫЕ РЕКОМЕНДАЦИИ
-----
  Город: Moscow
  Риск: 0.00
  Рекомендация: Погода благоприятная

7 ☐ БРОНИРОВАНИЯ ГОСТЯ
-----
  Найдено бронирований: 1
  • #1: Иван Петров, 15000.00 руб., active
=====

```

```

=====
✓ ВСЕ ОПЕРАЦИИ ВЫПОЛНЕННЫ УСПЕШНО
=====

```

```

▣ ИТОГ:
  • Добавлено номеров: 3
  • Создано бронирований: 2
  • Отменено бронирований: 1
  • Активных бронирований: 1

```

```

▣ Работа программы завершена

```

Система бронирования отелей

Ответы на контрольные вопросы:

В1. Чем отличается адаптер от репозитория в контексте сервисного слоя?

Репозиторий - это абстракция над хранилищем данных (БД, файлы). Он предоставляет методы для CRUD операций и поиска, скрывая детали реализации хранения.

Адаптер - это паттерн, который преобразует интерфейс одного класса в интерфейс, ожидаемый другим классом. В контексте сервисного слоя адаптер обычно используется для интеграции с внешними системами (API, очереди сообщений).

B2. Как реализовать транзакционность, если репозитории используют разные хранилища (SQL + NoSQL)?

Для обеспечения транзакционности при работе с разными хранилищами используется паттерн **Saga** (распределённая транзакция):

Варианты реализации:

1. Паттерн Saga (компенсирующие операции):

```
python
class HotelBookingService:
    def create_booking_with_payment(self, booking_data, payment_data):
        try:
            # Шаг 1: Создать бронирование в SQL
            booking = self.booking_repo.create(booking_data)

            # Шаг 2: Зарезервировать номер в NoSQL (Redis)
            self.room_repo.reserve(booking.room_id, booking.check_in, booking.check_out)

            # Шаг 3: Обработать платеж через внешний API
            payment = self.payment_adapter.charge(payment_data)

            return booking

        except PaymentFailedError:
            # Компенсирующие операции (откат)
            self.booking_repo.delete(booking.id) # Отмена в SQL
            self.room_repo.release(booking.room_id) # Освобождение в Redis
            raise

        except Exception as e:
            # Логирование и откат
            self._rollback(booking)
            raise
```

2. Использование двухфазной фиксации (2PC):

```
python
# Только если оба хранилища поддерживают 2PC
class TransactionManager:
    def execute_transaction(self, operations):
        # Подготовка
        for repo in self.repositories:
            repo.begin_transaction()

        try:
            # Выполнение операций
            results = [op() for op in operations]
            # Фиксация
            for repo in self.repositories:
                repo.commit()
            return results
        except Exception:
            # Откат
            for repo in self.repositories:
                repo.rollback()
            raise
```

В3. Как внедрить кеширование в сервис, не нарушая принцип единственной ответственности?

Использовать паттерн Декоратор для обёртки сервиса:

```
python
from functools import wraps
import json

class CachedHotelBookingService:
    """Декоратор для кеширования результатов сервиса."""

    def __init__(self, service, cache_backend):
        self._service = service
        self._cache = cache_backend # Redis, Memcached

    def search_available_rooms(self, search_params):
        # Создаем ключ кеша из параметров
        cache_key = f'rooms_search_{hash(str(search_params.__dict__))}'

        # Проверяем кеш
        cached = self._cache.get(cache_key)
        if cached:
            return json.loads(cached)

        # Получаем данные из сервиса
        result = self._service.search_available_rooms(search_params)

        # Сохраняем в кеш
        self._cache.set(cache_key, json.dumps([r.__dict__ for r in result]), ttl=300)
        return result

    # Делегируем остальные методы без кеширования
    def __getattr__(self, name):
        return getattr(self._service, name)

# Использование
service = HotelBookingService(room_repo, booking_repo)
cached_service = CachedHotelBookingService(service, redis_client)

# Основной сервис не знает о кешировании - SRP соблюден!
```

В4. Какие инструменты Python помогают реализовать Dependency Injection (кроме ручного)?

1. **FastAPI Depends** - встроенный DI для веб-приложений:

```
python
from fastapi import Depends, FastAPI

app = FastAPI()

def get_booking_service():
    return HotelBookingService(room_repo, booking_repo)

@app.get("/bookings/{booking_id}")
def get_booking(booking_id: int, service: HotelBookingService = Depends(get_booking_service)):
    return service.get_booking_details(booking_id)
```

2. **python-dependency-injector** - мощный фреймворк DI:

```
python
from dependency_injector import containers, providers

class Container(containers.DeclarativeContainer):
    room_repo = providers.Singleton(InMemoryRoomRepository)
    booking_repo = providers.Singleton(InMemoryBookingRepository)
    weather_service = providers.Singleton(WeatherService)

    booking_service = providers.Factory(
        HotelBookingService,
        room_repository=room_repo,
        booking_repository=booking_repo,
        weather_service=weather_service
    )

container = Container()
service = container.booking_service()
```

3. **Injector** (Google's dependency injector):

```
python
from injector import inject, Module, singleton

class AppModule(Module):
    def configure(self, binder):
        binder.bind(RoomRepository, to=InMemoryRoomRepository, scope=singleton)

class HotelBookingService:
    @inject
    def __init__(self, room_repo: RoomRepository):
        self.room_repo = room_repo
```

4. **Spring Python** - порт Spring Framework для Python

5. **Flask-Injector** - интеграция Injector с Flask

B5. Почему в сервисном слое не рекомендуется использовать ORM-сущности напрямую (например, SQLAlchemy модели)?

Причины:

1. Нарушение принципа разделения слоёв:

```
python
# ПЛОХО - сервис знает о SQLAlchemy
from sqlalchemy.orm import Session
```

```
class BookingService:
    def get_booking(self, booking_id: int, db: Session):
        return db.query(BookingModel).filter(BookingModel.id == booking_id).first()
```

2. Сложность тестирования:

ORM-сущности требуют подключения к БД

Нужно создавать тестовую БД или мокать сложные запросы

3. Проблемы при смене БД или хранилища:

Если перейти на NoSQL или файловое хранилище - придётся переписывать весь сервис

4. Утечка деталей реализации:

```
python
# ПЛОХО - ORM-сущность в сервисе
class BookingService:
    def get_bookings(self, user_id: int):
        # Сервис знает о ленивой загрузке, отношениях, session
        user = db.query(User).filter(User.id == user_id).first()
        return user.bookings # Ленивая загрузка!

# ХОРОШО - использование DTO (Data Transfer Object)
class BookingDTO:
    id: int
    guest_name: str
    total_price: Decimal

class BookingService:
    def get_bookings(self, user_id: int) -> List[BookingDTO]:
        # Сервис работает с DTO, не знает об ORM
        models = self.repo.get_by_user(user_id)
        return [BookingDTO(**model.__dict__) for model in models]
```

5. Сложность миграций:

Изменения в ORM-моделях требуют изменений в сервисе

В6. Как протестировать сервис, если он вызывает внешний API (например, платёжную систему)?

Использование моков и патчей:

```
python
import pytest
from unittest.mock import Mock, patch
from decimal import Decimal

class TestPaymentService:

    def test_successful_payment(self):
        # Создаем мок для внешнего API
        mock_payment_api = Mock()
        mock_payment_api.charge.return_value = {
            'status': 'success',
            'transaction_id': 'tx_123456'
        }

        # Внедряем мок через DI
        service = PaymentService(payment_adapter=mock_payment_api)

        # Выполняем тест
        result = service.process_payment(
            booking_id=1,
            amount=Decimal('15000.00'),
            card_data={'number': '4111111111111111', 'expiry': '12/25'}
        )

        # Проверяем результат
        assert result['status'] == 'success'
        assert result['transaction_id'] == 'tx_123456'
```

```

# Проверяем, что API был вызван с правильными параметрами
mock_payment_api.charge.assert_called_once_with(
    amount=Decimal('15000.00'),
    card_data={'number': '4111111111111111', 'expiry': '12/25'}
)

def test_payment_failure(self):
    # Мок с ошибкой
    mock_payment_api = Mock()
    mock_payment_api.charge.side_effect = PaymentFailedError("Insufficient funds")

    service = PaymentService(payment_adapter=mock_payment_api)

    with pytest.raises(PaymentFailedError) as exc_info:
        service.process_payment(booking_id=1, amount=Decimal('100'), card_data={})

    assert "Insufficient funds" in str(exc_info.value)

@patch('requests.post')
def test_with_patch(self, mock_post):
    # Использование patch для замены requests.post
    mock_post.return_value.json.return_value = {'status': 'success'}
    mock_post.return_value.status_code = 200

    service = PaymentService()
    result = service.process_payment(booking_id=1, amount=Decimal('100'), card_data={})

    assert result['status'] == 'success'
    mock_post.assert_called_once()

```

В7. В чём разница между валидацией на уровне DTO (схем) и валидацией в сервисе?

Валидация на уровне DTO (синтаксическая):

Проверка формата данных

Проверка типов, обязательных полей

Проверка длины строк, формата email, телефона

Выполняется до попадания в сервис

Валидация в сервисе (бизнес-логическая):

Проверка бизнес-правил

Проверка существования сущностей в БД

Проверка доступности ресурсов

Проверка прав доступа

В8. Как логировать ошибки так, чтобы сохранить контекст (user_id, заказ)?

Использование структурированного логирования:

```

python
import logging
import json
from typing import Optional

class ContextLogger:
    """Логгер с контекстом."""

```

```

def __init__(self, name: str = __name__):
    self.logger = logging.getLogger(name)
    self._context = {}

def set_context(self, **kwargs):
    """Установка контекста для всех последующих логов."""
    self._context.update(kwargs)

def add_context(self, **kwargs):
    """Добавление контекста для одного лога."""
    return {**self._context, **kwargs}

def info(self, message: str, **kwargs):
    self.logger.info(message, extra={'context': self.add_context(**kwargs)})

def error(self, message: str, exception: Optional[Exception] = None, **kwargs):
    extra = {'context': self.add_context(**kwargs)}
    if exception:
        extra['exc_info'] = True
    self.logger.error(message, extra=extra)

```

Использование в сервисе

```
class BookingService:
```

```
    def __init__(self):
        self.logger = ContextLogger(__name__)
```

```
    def create_booking(self, booking_data):
        # Устанавливаем контекст
        self.logger.set_context(
            user_id=booking_data.guest_email,
            action='create_booking'
        )
```

```
    try:
        # Логим успешную операцию
        self.logger.info(
            f"Creating booking for {booking_data.guest_name}",
            room_id=booking_data.room_id,
            check_in=booking_data.check_in.isoformat()
        )
```

```
        booking = self._create_booking_impl(booking_data)
```

```
        self.logger.info(
            "Booking created successfully",
            booking_id=booking.id,
            total_price=float(booking.total_price)
        )
        return booking
```

```
    except RoomNotFoundError as e:
        self.logger.error(
            f"Room not found: {e}",
            exception=e,
            room_id=booking_data.room_id
        )
```

```

        raise

    except Exception as e:
        self.logger.error(
            f'Booking creation failed: {e}',
            exception=e,
            guest_email=booking_data.guest_email
        )
        raise

# Форматирование логов в JSON
class JSONFormatter(logging.Formatter):
    def format(self, record):
        log_data = {
            'timestamp': datetime.now().isoformat(),
            'level': record.levelname,
            'logger': record.name,
            'message': record.getMessage()
        }
        if hasattr(record, 'context'):
            log_data['context'] = record.context
        if record.exc_info:
            log_data['exception'] = self.formatException(record.exc_info)
        return json.dumps(log_data, ensure_ascii=False)

```

В9. Как организовать «мягкое удаление» записей через сервисный слой?

Реализация мягкого удаления:

```

python
from dataclasses import dataclass
from datetime import datetime
from typing import Optional

# 1. Добавляем поле deleted_at в модель
@dataclass
class Room:
    id: int
    hotel_name: str
    room_number: str
    # ... другие поля
    deleted_at: Optional[datetime] = None

    @property
    def is_deleted(self) -> bool:
        return self.deleted_at is not None

# 2. Репозиторий с учетом мягкого удаления
class RoomRepository:
    def get_by_id(self, room_id: int, include_deleted: bool = False) -> Optional[Room]:
        if include_deleted:
            return self._rooms.get(room_id)

        room = self._rooms.get(room_id)
        if room and room.deleted_at is None:
            return room
        return None

    def get_all(self, include_deleted: bool = False) -> List[Room]:

```

```

rooms = list(self._rooms.values())
if not include_deleted:
    rooms = [r for r in rooms if r.deleted_at is None]
return rooms

def soft_delete(self, room_id: int) -> Room:
    room = self._rooms.get(room_id)
    if not room:
        raise RoomNotFoundError(room_id)

    # Проверяем, что номер можно удалить
    if self._has_active_bookings(room_id):
        raise BusinessRuleViolationError("Номер имеет активные бронирования")

    room.deleted_at = datetime.now()
    return room

def restore(self, room_id: int) -> Room:
    room = self._rooms.get(room_id)
    if not room:
        raise RoomNotFoundError(room_id)

    if room.deleted_at is None:
        raise BusinessRuleViolationError("Номер не был удален")

    room.deleted_at = None
    return room

# 3. Сервисный слой с мягким удалением
class HotelBookingService:
    def delete_room(self, room_id: int) -> Room:
        """Мягкое удаление номера."""
        logger.info(f"Мягкое удаление номера: {room_id}")

        # Проверяем бизнес-правила
        room = self.room_repo.get_by_id(room_id, include_deleted=False)
        if not room:
            raise RoomNotFoundError(room_id)

        # Проверяем активные бронирования
        active_bookings = self.booking_repo.get_active_for_room(room_id)
        if active_bookings:
            raise BusinessRuleViolationError(
                f"Номер {room_id} имеет {len(active_bookings)} активных бронирований"
            )

        # Выполняем мягкое удаление
        result = self.room_repo.soft_delete(room_id)
        logger.info(f"Номер {room_id} мягко удален в {result.deleted_at}")
        return result

    def restore_room(self, room_id: int) -> Room:
        """Восстановление мягко удаленного номера."""
        logger.info(f"Восстановление номера: {room_id}")

        room = self.room_repo.get_by_id(room_id, include_deleted=True)
        if not room:

```



```

        raise RoomNotFoundError(room_id)

    if room.deleted_at is None:
        raise BusinessRuleViolationError("Номер не был удален")

    result = self.room_repo.restore(room_id)
    logger.info(f"Номер {room_id} восстановлен")
    return result

    def get_all_rooms(self, include_deleted: bool = False) -> List[Room]:
        """Получить все номера с возможностью включить удаленные."""
        return self.room_repo.get_all(include_deleted=include_deleted)

```

V10. Приведите пример пагинации в репозитории и использования в сервисе.

Реализация пагинации:

```

python
from dataclasses import dataclass
from typing import List, Generic, TypeVar, Optional

```

```

T = TypeVar('T')

```

```

@dataclass
class PageResult(Generic[T]):
    """Результат пагинированного запроса."""
    items: List[T]
    total: int
    page: int
    page_size: int
    total_pages: int
    has_next: bool
    has_previous: bool

```

1. Репозиторий с пагинацией

```

class InMemoryBookingRepository:
    def get_by_guest_email(
        self,
        email: str,
        page: int = 1,
        page_size: int = 10
    ) -> PageResult[Booking]:
        """Получить бронирования гостя с пагинацией."""
        with self._lock:
            # Получаем все бронирования
            bookings = [
                self._bookings[bid]
                for bid in self._guest_bookings.get(email, [])
                if bid in self._bookings
            ]

            total = len(bookings)
            total_pages = (total + page_size - 1) // page_size if total > 0 else 1

            # Проверка границ
            if page < 1:
                page = 1

```

```

    if page > total_pages:
        page = total_pages

    # Вычисляем срез
    start = (page - 1) * page_size
    end = min(start + page_size, total)

    return PageResult(
        items=bookings[start:end],
        total=total,
        page=page,
        page_size=page_size,
        total_pages=total_pages,
        has_next=page < total_pages,
        has_previous=page > 1
    )

def get_active_bookings(
    self,
    page: int = 1,
    page_size: int = 20,
    status_filter: Optional[BookingStatus] = None
) -> PageResult[Booking]:
    """Получить активные бронирования с пагинацией и фильтрацией."""
    with self._lock:
        bookings = list(self._bookings.values())

    # Фильтрация
    if status_filter:
        bookings = [b for b in bookings if b.status == status_filter]

    # Сортировка по дате создания (новые сверху)
    bookings.sort(key=lambda b: b.created_at, reverse=True)

    total = len(bookings)
    total_pages = (total + page_size - 1) // page_size if total > 0 else 1

    if page < 1:
        page = 1
    if page > total_pages:
        page = total_pages

    start = (page - 1) * page_size
    end = min(start + page_size, total)

    return PageResult(
        items=bookings[start:end],
        total=total,
        page=page,
        page_size=page_size,
        total_pages=total_pages,
        has_next=page < total_pages,
        has_previous=page > 1
    )

```

2. Сервис с пагинацией

```

class HotelBookingService:

```

```

def get_guest_bookings_paginated(
    self,
    email: str,
    page: int = 1,
    page_size: int = 10
) -> PageResult[Booking]:
    """Получить бронирования гостя с пагинацией."""
    logger.info(f"Получение бронирований гостя {email}, страница {page}, размер {page_size}")

    if page < 1:
        raise ValidationError("Номер страницы должен быть >= 1")

    if page_size < 1 or page_size > 100:
        raise ValidationError("Размер страницы должен быть от 1 до 100")

    result = self.booking_repo.get_by_guest_email(email, page, page_size)

    logger.info(f"Найдено {result.total} бронирований, страница {result.page}/{result.total_pages}")
    return result

def search_rooms_paginated(
    self,
    search_params: RoomSearch,
    page: int = 1,
    page_size: int = 20
) -> PageResult[Room]:
    """Поиск номеров с пагинацией."""
    # Получаем все подходящие номера
    all_rooms = self.search_available_rooms(search_params)

    total = len(all_rooms)
    total_pages = (total + page_size - 1) // page_size if total > 0 else 1

    if page < 1:
        page = 1
    if page > total_pages:
        page = total_pages

    start = (page - 1) * page_size
    end = min(start + page_size, total)

    return PageResult(
        items=all_rooms[start:end],
        total=total,
        page=page,
        page_size=page_size,
        total_pages=total_pages,
        has_next=page < total_pages,
        has_previous=page > 1
    )

```

3. Использование в клиентском коде

```

def main():
    service = HotelBookingService(room_repo, booking_repo)

    # Получение с пагинацией
    page_1 = service.get_guest_bookings_paginated("ivan@mail.ru", page=1, page_size=5)

```

```

print(f"Всего бронирований: {page_1.total}")
print(f"Страница {page_1.page} из {page_1.total_pages}")
for booking in page_1.items:
    print(f" #{booking.id}: {booking.guest_name}")

# Проверка наличия следующих страниц
if page_1.has_next:
    print("Есть еще страницы")

# Получение следующей страницы
page_2 = service.get_guest_bookings_paginated(
    "ivan@mail.ru",
    page=page_1.page + 1,
    page_size=5
)
print(f"Страница {page_2.page} из {page_2.total_pages}")

# Поиск номеров с пагинацией
search = RoomSearch(hotel_name="Grand", max_guests=2)
rooms_page = service.search_rooms_paginated(search, page=1, page_size=10)

print(f"Найдено номеров: {rooms_page.total}")
for room in rooms_page.items:
    print(f" {room.hotel_name} - {room.room_number}")

```

Вывод: В ходе выполнения практического занятия была успешно разработана сервисная часть системы бронирования отелей. Все поставленные задачи выполнены в полном объеме:

- Разработана многослойная архитектура приложения.
- Реализованы паттерны Repository, Service Layer, Dependency Injection.
- Создана иерархия собственных исключений для обработки бизнес-ошибок.
- Реализована бизнес-логика: добавление номеров, поиск, бронирование, отмена со штрафами.
- Обеспечено разделение бизнес-логики и инфраструктурного кода.
- Реализовано логирование всех ключевых операций.
- Продемонстрирована интеграция с внешним сервисом (погода).

Разработанная система демонстрирует правильное применение архитектурных паттернов и принципов SOLID. Архитектура обеспечивает гибкость, тестируемость и расширяемость, что позволяет в будущем легко

добавлять новый функционал, подключать реальные базы данных и реализовывать REST API.

Работа позволила на практике освоить принципы построения сервисного слоя, что является важным навыком для разработки качественных и поддерживаемых программных продуктов. Полученные знания и навыки могут быть применены при разработке любых приложений со сложной бизнес-логикой.